

Nome: Emanuelli Marques de Souza

Professor: Jackson

Título: Pesquisa sobre APIs, HTTP, Rotas, REST e Express

Curso: Técnico em Desenvolvimento de Sistemas

Palhoça
2026

Diagrama



Nesse exemplo, o cliente solicita o usuário de ID 15. O servidor procura esse usuário e retorna uma resposta com o código **200 OK** e os dados encontrados.

PARTE III — O que é uma rota?

Uma rota é um endereço que uma API disponibiliza para que o cliente possa acessar determinado recurso ou realizar uma operação.

A rota serve para indicar ao servidor qual recurso está sendo solicitado e qual operação deve ser realizada.

Uma rota é identificada principalmente pelo **método HTTP** junto com o **caminho da URL**.

Por exemplo:

```
GET /users
```

e

`POST /users` possuem o mesmo caminho, mas representam operações diferentes.

A relação entre rota e recurso é que a rota normalmente representa um recurso que a API administra.

Uma API possui várias rotas porque pode trabalhar com vários recursos diferentes e realizar operações diferentes sobre eles.

PARTE VI — Parâmetros de rota

Considere:

```
GET /users/15
```

`/users` representa o recurso **usuários**.

O número 15 representa o **identificador** de um usuário específico.

Utilizar um identificador na URL é importante porque permite indicar exatamente qual recurso queremos acessar.

Por exemplo:

```
/users/10  
/users/15  
/users/20
```

Cada URL representa um usuário diferente.

O servidor pode receber o número 15 e procurar no banco de dados ou em um array o usuário que possui esse ID.

Path parameter

O **path parameter** faz parte do caminho da URL.

Exemplos:

```
/users/15  
/products/5
```

Nesse caso, 15 e 5 são parâmetros de rota.

Query parameter

O **query parameter** aparece depois do `?`.

Exemplos:

```
/users?id=15  
/products?category=books
```

400 — Bad Request

Significa que a requisição possui algum problema.

Exemplo: o cliente tenta cadastrar um usuário sem enviar um campo obrigatório.

401 — Unauthorized

Indica que é necessário realizar autenticação ou que a autenticação não é válida.

Exemplo: uma pessoa tenta acessar uma área protegida sem estar autorizada.

403 — Forbidden

Significa que o servidor entendeu a requisição, mas não permite aquela operação.

Exemplo: um usuário comum tenta acessar uma função exclusiva de administrador.

404 — Not Found

Significa que o recurso solicitado não foi encontrado.

Exemplo:

```
GET /students/999
```

se o estudante 999 não existir.

500 — Internal Server Error

Significa que ocorreu um erro inesperado no servidor.

Exemplo: o servidor apresenta uma falha durante o processamento da requisição.

Questão para reflexão

É importante utilizar constantemente os códigos de status porque eles ajudam o cliente a entender o que aconteceu com a requisição. Dessa forma, o sistema pode saber se a operação deu certo, se os dados estavam incorretos, se o recurso não existe ou se houve um problema no servidor.

1. Introdução

Atualmente, muitos sistemas e aplicativos precisam trocar informações entre si. Para que essa comunicação aconteça de forma organizada, são utilizadas as APIs. Elas estão presentes em aplicativos de bancos, lojas virtuais, sistemas escolares, aplicativos de transporte, redes sociais e muitos outros sistemas.

Nesta pesquisa serão apresentados os principais conceitos relacionados às APIs, como cliente, servidor, requisições HTTP, respostas, rotas, métodos HTTP, parâmetros, JSON, códigos de status, REST e Express. Também será apresentado o projeto de uma API para uma biblioteca.

PARTE I — Entendendo o conceito de API

5.1 O que é uma API?

API significa **Application Programming Interface**, que em português significa **Interface de Programação de Aplicações**.

Com muitas palavras, uma API é uma forma de comunicação entre diferentes sistemas. Ela permite que um programa solicite informações ou envie dados para outro sistema de uma maneira organizada.

Uma API serve para permitir essa comunicação sem que um sistema precise conhecer todo o funcionamento interno do outro. Por exemplo, um aplicativo pode pedir informações para uma API e receber os dados necessários como resposta.

Uma aplicação pode precisar de uma API quando precisa utilizar informações ou serviços que estão em outro sistema. Um aplicativo de celular, por exemplo, pode precisar consultar um servidor para buscar informações de usuários, produtos ou outros dados.

Vários sistemas podem consumir uma API, como sites, aplicativos de celular, sistemas escolares, lojas virtuais, aplicativos bancários, sistemas de transporte e outros programas.

5.2 API no mundo real

Exemplo 1 — Aplicativo de previsão do tempo

Um aplicativo de previsão do tempo pode utilizar uma API para buscar informações sobre o clima.

Exemplos

`/users` representa usuários.

`/products` representa produtos.

`/orders` representa pedidos.

`/students` representa estudantes.

`/books` representa livros.

Por exemplo:

```
GET /books
```

poderia servir para listar livros.

PARTE IV — Métodos HTTP

Os métodos HTTP indicam qual operação o cliente deseja realizar:

GET

O GET é utilizado principalmente para **consultar** ou **buscar informações**. Uma situação comum seria listar produtos:

`GET /products`

Normalmente não envia dados no body e normalmente retorna informações para o cliente.

POST

O POST é utilizado principalmente para **criar um novo recurso** ou **enviar dados** para o servidor. Por exemplo, para cadastrar um usuário.

`POST /users`

Normalmente envia informações no body e pode retornar o recurso que foi criado.

PUT

O PUT é utilizado para **atualizar** ou **substituir** um recurso.

A principal diferença é que o path parameter normalmente identifica um recurso específico, enquanto o query parameter é utilizado principalmente para filtros, pesquisas e outras opções de consulta.

PARTE VII — Query parameters

Query parameters são parâmetros utilizados no final da URL para passar informações adicionais para uma consulta.

Eles podem ser utilizados para:

- filtros;
- pesquisas;
- paginação;
- ordenação;
- limite de resultados.

Por exemplo:

```
GET /products?category=books
```

pode buscar somente produtos da categoria de livros.

Outro exemplo:

```
GET /products?page=2&limit=10
```

pode indicar que queremos a segunda página com no máximo 10 produtos.

A diferença para um parâmetro de rota é que:

```
/products/10
```

normalmente identifica diretamente o produto de ID 10.

Já

```
/products?category=books
```

funciona como uma consulta ou filtro.

Três exemplos próprios

PARTE X — REST e APIs RESTful

REST significa **Representational State Transfer**. É um estilo de arquitetura utilizado para organizar sistemas que se comunicam através de uma rede.

Uma API RESTful é uma API que segue os princípios do REST.

Os recursos são as entidades que a API administra, como estudantes, livros, produtos ou usuários.

Existe uma relação entre recurso, URL e métodos HTTP. A URL identifica o recurso e o método HTTP indica a operação que será realizada.

Por exemplo:

```
GET /students
```

lista estudantes.

`POST /students`

cadasta um estudante.

```
GET /students/10
```

busca o estudante de ID 10.

```
PUT /students/10
```

atualiza o estudante 10.

```
DELETE /students/10
```

exclui o estudante 10.

Esse conjunto é organizado porque utiliza o mesmo recurso, `students`, enquanto os métodos HTTP indicam as diferentes operações.

PARTE XI — O papel do Express

Quem seria o cliente?

O cliente seria o aplicativo de previsão do tempo instalado no celular.

Quem forneceria os dados?

Os dados seriam fornecidos por um servidor ou serviço especializado em informações meteorológicas.

Que informações poderiam ser solicitadas?

Poderiam ser solicitadas informações como temperatura, umidade, velocidade do vento, previsão de chuva e condições climáticas.

Exemplo 2 — Loja virtual

Uma loja virtual também pode utilizar uma API para buscar e administrar seus produtos.

Quem seria o cliente?

O cliente seria o site ou aplicativo da loja.

Quem forneceria os dados?

O servidor de loja, que pode buscar os dados em um banco de dados.

Que informações poderiam ser solicitadas?

Poderiam ser solicitados nome dos produtos, preços, categorias, quantidade em estoque e informações sobre pedidos.

PARTE II — Cliente, servidor e requisições

1. O que é um cliente?

O cliente é o sistema que inicia uma comunicação com o servidor fazendo uma requisição. Pode ser um navegador, aplicativo de celular, site ou outro programa.

2. O que é um servidor?

O servidor é o sistema responsável por receber as requisições dos clientes, processá-las e enviar respostas.

Ele pode consultar um banco de dados, cadastrar informações, atualizar dados ou realizar outras operações antes de responder.

3. O que acontece quando um cliente faz uma requisição?

Por exemplo:

```
PUT /users/10
```

Normalmente envia os novos dados no body.

PATCH

O PATCH é utilizado para **alterar apenas uma parte de um recurso**.

Por exemplo, se queremos alterar somente o email:

```
PATCH /users/10
```

Normalmente envia os dados que serão modificados no body.

DELETE

O DELETE é utilizado para **excluir um recurso**.

Por exemplo:

```
DELETE /users/10
```

Normalmente não precisa enviar dados no body e pode retornar apenas uma confirmação da operação.

Tabela

Método	Finalidade	Exemplo
GET	Consultar dados	<code>GET /users/5</code>
POST	Criar um recurso	<code>POST /users</code>
PUT	Atualizar/substituir	<code>PUT /users/10</code>
PATCH	Alterar parcialmente	<code>PATCH /users/10</code>
DELETE	Excluir um recurso	<code>DELETE /users/10</code>

PARTE V — Quando criar uma nova rota?

```
GET /movies?genre=action
```

Busca filmes de ação.

```
GET /students?name=Maria
```

Pesquisa estudantes pelo nome.

```
GET /books?page=2&limit=5
```

Busca a segunda página de livros, mostrando até cinco resultados.

PARTE VIII — Corpo da requisição

A requisição apresentada é:

```
POST /users  
Content-Type: application/json
```

```
{  
  "name": "Maria",  
  "email": "maria@email.com"  
}
```

Nesse caso, estão sendo enviados para o servidor o nome e o email da pessoa.

JSON significa **JavaScript Object Notation**. É um formato utilizado para representar dados de maneira organizada e é muito utilizado na comunicação entre APIs e aplicações.

O corpo da requisição é utilizado para enviar dados ao servidor. Ele é muito comum quando queremos criar ou alterar informações.

O envio de dados no body é comum principalmente nos métodos:

- POST;
- PUT;
- PATCH.

A diferença entre dados enviados pela URL e pelo body é que a URL, normalmente, identifica um recurso ou fornece opções para uma consulta, enquanto o body pode conter uma quantidade maior de informações que serão cadastradas ou alteradas.

O que é o Express?

O Express é um framework utilizado com Node.js para facilitar a criação de servidores e APIs.

O Node.js permite executar JavaScript no lado do servidor. O Express funciona sobre o Node.js e fornece ferramentas que facilitam a criação de rotas, requisições e respostas.

Ele é utilizado para criar servidores e APIs porque simplifica bastante a estrutura do código.

Com Express, podemos criar uma rota utilizando:

```
app.get('/users', (req, res) => {  
  res.send('Lista de usuários')  
})
```

Nesse exemplo, quando o cliente faz uma requisição `GET /users`, o Express encaminha a função.

Também podemos utilizar:

```
app.post('/users', (req, res) => {  
  res.send('Usuário cadastrado')  
})
```

```
app.put('/users/id', (req, res) => {  
  res.send('Usuário atualizado')  
})
```

```
app.patch('/users/id', (req, res) => {  
  res.send('Usuário alterado')  
})
```

```
app.delete('/users/id', (req, res) => {  
  res.send('Usuário excluído')  
})
```

Esses exemplos mostram como os conceitos de métodos HTTP e rotas aparecem diretamente no código.

PARTE XII — Projetando uma API

Primeiro, o cliente envia uma requisição para o servidor. Essa requisição informa o que o cliente deseja fazer.

O servidor recebe a requisição, verifica as informações enviadas, realiza o processamento necessário e depois envia uma resposta para o cliente.

4. O que é uma resposta?

Uma resposta é a mensagem enviada pelo servidor depois de processar uma requisição.

Ela pode informar se a operação deu certo ou se ocorreu algum erro. Também pode trazer dados solicitados pelo cliente.

5. O que poderia existir dentro de uma requisição HTTP?

Uma requisição HTTP pode conter:

- método HTTP;
- URL;
- parâmetros;
- cabeçalhos;
- corpo da requisição, quando necessário.

Por exemplo:

```
GET /users/15
```

6. O que poderia existir dentro de uma resposta HTTP?

Uma resposta HTTP pode conter:

- código de status;
- cabeçalhos;
- dados da resposta.

Por exemplo:

```
200 OK
```

```
{  
  "id": 15,  
  "name": "Maria"  
}
```

Faz sentido criar uma nova rota quando a API precisa disponibilizar um novo recurso ou um endpoint com uma finalidade diferente.

Uma rota de uma API não representa necessariamente uma página. Ela normalmente representa um **recurso**.

Um recurso é algo que o sistema administra, como:

- usuários;
- produtos;
- livros;
- pedidos;
- estudantes.

Uma ação é aquilo que fazemos com o recurso, como consultar, cadastrar, atualizar ou excluir.

Não é necessário criar uma rota para cada ação realizada pelo usuário. Os métodos HTTP já podem representar diferentes operações sobre o mesmo recurso.

```
GET /users
```

Pode ser utilizado para listar os usuários.

```
POST /users
```

Pode ser utilizado para cadastrar um novo usuário.

```
GET /users/10
```

Pode ser utilizado para buscar o usuário de ID 10.

```
PUT /users/10
```

Pode ser utilizado para atualizar o usuário de ID 10.

```
DELETE /users/10
```

Pode ser utilizado para excluir o usuário de ID 10.

Questão para reflexão

Não precisamos criar uma rota diferente para cada operação porque os métodos HTTP já conseguem representar as operações realizadas sobre um mesmo recurso.

Por exemplo, `/users` pode ser usado com GET para consultar e com POST para cadastrar. Dessa forma, a API fica mais organizada e padronizada.

Por exemplo:

```
GET /users/15
```

utiliza a URL para identificar o usuário.

```
Já
```

```
POST /users
```

pode utilizar um body:

```
{  
  "name": "Maria",  
  "email": "maria@email.com"  
}
```

para enviar os dados do novo usuário.

PARTE IX — Respostas HTTP e códigos de status

200 — OK

Significa que a requisição foi realizada com sucesso.

Exemplo: `GET /users` encontrou os usuários e retornou os dados.

201 — Created

Significa que um novo recurso foi criado.

Exemplo: depois de `POST /users`, um novo usuário foi cadastrado.

204 — No Content

Significa que a operação foi realizada com sucesso, mas não existe conteúdo para retornar.

Exemplo: um produto foi excluído e o servidor não precisa enviar dados na resposta.

Para o sistema de biblioteca, serão utilizados os recursos **livros, autores, usuários e empréstimos**.

A API poderia possuir as seguintes rotas:

Método	Rota	Descrição
GET	<code>/livros</code>	Listar todos os livros
GET	<code>/livros/:id</code>	Buscar um livro específico
POST	<code>/livros</code>	Cadastrar um livro
PUT	<code>/livros/:id</code>	Atualizar um livro
DELETE	<code>/livros/:id</code>	Excluir um livro
GET	<code>/autores</code>	Listar autores
GET	<code>/autores/:id</code>	Buscar um autor específico
POST	<code>/autores</code>	Cadastrar um autor
PUT	<code>/autores/:id</code>	Atualizar um autor
DELETE	<code>/autores/:id</code>	Excluir um autor
GET	<code>/usuarios</code>	Listar usuários
GET	<code>/usuarios/:id</code>	Buscar um usuário
POST	<code>/usuarios</code>	Cadastrar um usuário
GET	<code>/emprestos</code>	Listar empréstimos
POST	<code>/emprestos</code>	Registrar um empréstimo

A API possui quatro recursos e mais de dez rotas.

PARTE XIII — Pensando como um desenvolvedor

Situação 1

Eu utilizo:

```
GET /products?name=notebook
```

Eu encontro essa opção porque continuamos trabalhando com o mesmo recurso, que são os produtos. O `name` seria utilizado como um filtro para pesquisar pelo nome.

Não seria necessário criar `GET /products/search`, pois a pesquisa pode ser feita utilizando um `query parameter`.

Situação 2

A melhor opção seria:

```
GET /users/25
```

Isso porque estamos procurando um usuário específico pelo seu ID. O número 25 identifica exatamente qual usuário queremos buscar.

Situação 3

Para cadastrar um novo produto, o método adequado é:

```
POST
```

Por exemplo:

```
POST /products
```

Os dados do novo produto poderiam ser enviados no `body`.

Situação 4

Para excluir o produto de ID 8, a requisição adequada seria:

```
DELETE /products/8
```

O `DELETE` é utilizado para excluir recursos e o número 8 identifica qual produto deve ser excluído.

Situação 5

Se o cliente solicitar:

```
GET /usuarios/999
```

e o estudante não existir, a API poderia retornar:

```
404 Not Found
```

Esse código indica que o recurso solicitado não foi encontrado.

PARTE XIV — Projeto final da pesquisa

18.1 Nome da API

API de Biblioteca

A API de Biblioteca serviria para organizar as informações de uma biblioteca. Ela permitiria cadastrar e consultar livros, autores e usuários, além de registrar os empréstimos realizados.

18.3 Recursos

Os principais recursos seriam:

- livros;
- autores;
- usuários;
- empréstimos.

18.4 Rotas

Método	Rota	Finalidade
GET	<code>/livros</code>	Listar livros
GET	<code>/livros/:id</code>	Buscar livro
POST	<code>/livros</code>	Cadastrar livro
PUT	<code>/livros/:id</code>	Atualizar livro
DELETE	<code>/livros/:id</code>	Excluir livro
GET	<code>/autores</code>	Listar autores
GET	<code>/autores/:id</code>	Buscar autor
POST	<code>/autores</code>	Cadastrar autor
GET	<code>/usuarios</code>	Listar usuários
GET	<code>/usuarios/:id</code>	Buscar usuário
POST	<code>/usuarios</code>	Cadastrar usuário
GET	<code>/emprestimos</code>	Listar empréstimos
POST	<code>/emprestimos</code>	Registrar empréstimo
DELETE	<code>/emprestimos/:id</code>	Excluir empréstimo

18.5 Exemplos de requisição

Exemplo 1 — Listar livros

Método: GET

URL:

```
GET /livros
```

Não seria necessário enviar `body`.

Uma possível resposta seria:

```
{
```

```
"id": 1,
"titulo": "Dom Casimiro",
"autor": "Machado de Assis"
},
{
  "id": 2,
  "titulo": "O Contigo",
  "autor": "Miguel Azavedo"
}
```

Código de status:

```
200 OK
```

Exemplo 2 — Cadastrar livro

Método: POST

URL:

```
POST /livros
```

Body:

```
{
  "titulo": "Dom Casimiro",
  "autor": "Machado de Assis",
  "ano": 1899
}
```

Uma possível resposta seria:

```
{
  "id": 10,
  "titulo": "Dom Casimiro",
  "autor": "Machado de Assis",
  "ano": 1899
}
```

Código de status:

```
201 Created
```

Exemplo 3 — Buscar livro

Método: GET

URL:

```
GET /livros/10
```

Nesse caso, 10 é o parâmetro de rota.

Uma possível resposta seria:

```
{
  "id": 10,
  "titulo": "Dom Casimiro",
  "autor": "Machado de Assis",
  "ano": 1899
}
```

Código de status:

```
200 OK
```

Se o livro não existir, poderia retornar:

```
404 Not Found
```

PARTE XV — Conclusão

Depois de realizar a pesquisa, entendi que uma API é uma forma de comunicação entre diferentes sistemas. Ela permite que um cliente envie uma requisição para um servidor e receba uma resposta com informações ou com o resultado de uma operação.

A função de uma rota é indicar um endereço que pode ser utilizado para acessar determinado recurso da API. Por exemplo, `/users` pode representar os usuários e `/products` pode representar os produtos.

Os métodos HTTP são importantes porque indicam qual operação queremos realizar. O `GET` é utilizado principalmente para consultar, o `POST` para criar, o `PUT` para atualizar ou substituir, o `PATCH` para fazer alterações parciais e o `DELETE` para excluir.

Para decidir qual método utilizar, é necessário observar o objetivo da requisição. Se quero consultar, utilizo `GET`. Se quero cadastrar, utilizo `POST`. Se quero atualizar, posso utilizar `PUT` ou `PATCH`. Se quero excluir, utilizo `DELETE`.

Uma nova rota deve ser criada quando for necessário representar um novo recurso ou endpoint. Não é necessário criar uma rota diferente para cada ação, porque podemos utilizar diferentes métodos HTTP no mesmo recurso.

O Express tem o papel de facilitar a criação de servidores e APIs utilizando Node.js. Com ele podemos criar rotas de forma simples utilizando métodos como `app.get()`, `app.post()`, `app.put()`, `app.patch()` e `app.delete()`.

Depois da pesquisa, minha compreensão sobre APIs mudou, porque passei a entender que uma API não é somente um código que recebe informações. Existe toda uma organização por trás dela, envolvendo cliente, servidor, rotas, métodos HTTP, parâmetros, `body`, respostas e códigos de status.

PARTE XVII — Referências

MDN Web Docs. HTTP request methods. Mozilla Developer Network. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>. Acesso em: 25 ago. 2026.

MDN Web Docs. HTTP response status codes. Mozilla Developer Network. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>. Acesso em: 25 ago. 2026.

Express.js. Routing. Express.js. Disponível em: <https://expressjs.com/en/guide/routing.html>. Acesso em: 25 ago. 2026.

Express.js. Basic routing. Express.js. Disponível em: <https://expressjs.com/en/4x/api.html#router-basic-routing>. Acesso em: 25 ago. 2026.

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/>. Acesso em: 25 ago. 2026.